

STUDY NOTE

init · systemd · 컨테이너 · 네트워크

01 init 시스템

02 systemd

03 컨테이너

04 네트워크

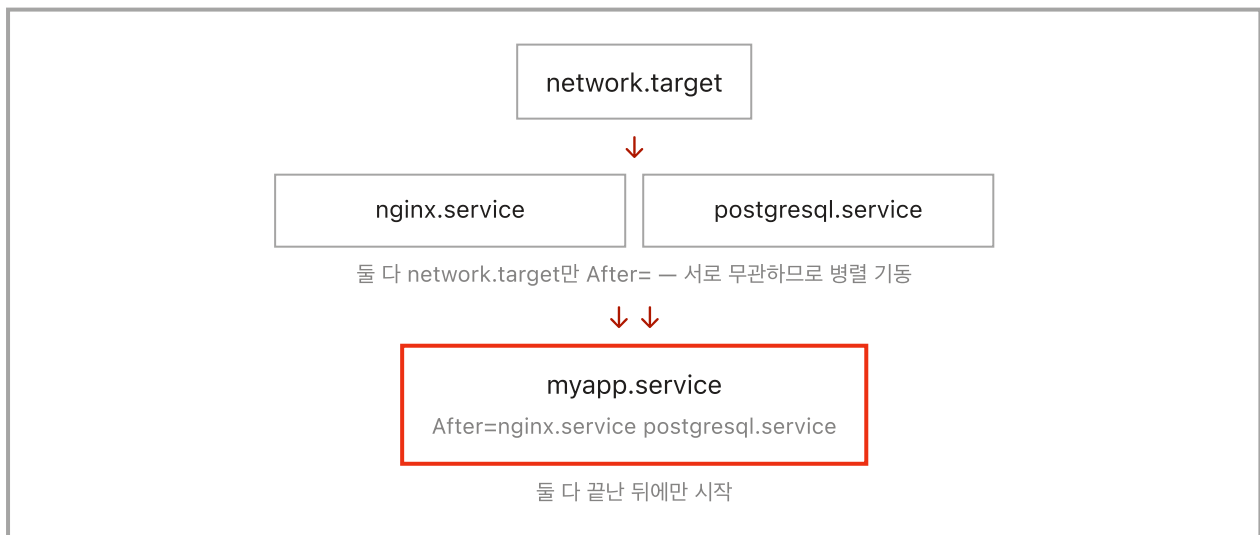
01

init 시스템이란

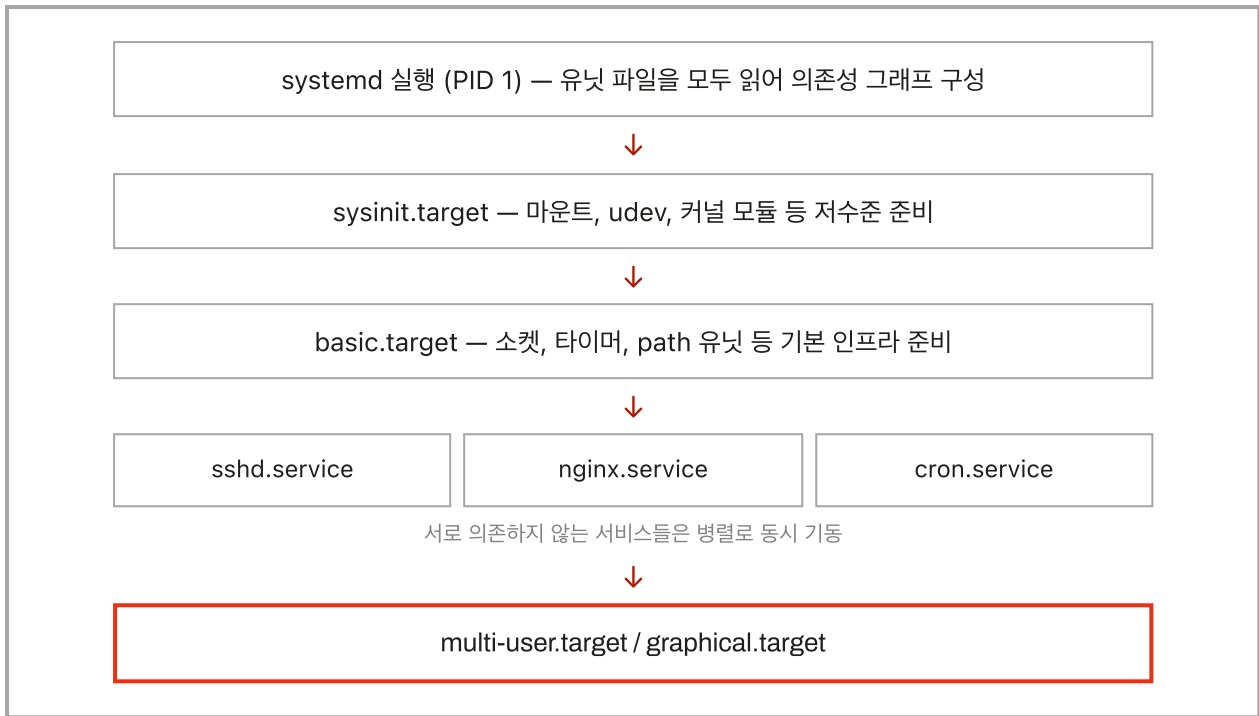
커널이 부팅을 마치면 가장 먼저 사용자 공간에서 **PID 1** 프로세스를 실행합니다. 이 프로세스가 "init"입니다. init은 이후 모든 프로세스의 조상이 되며, 부모를 잃은 고아 프로세스(orphan)를 입양해 좀비 프로세스가 쌓이지 않도록 정리하는 역할도 맡습니다. 오늘날 대부분의 배포판에서 이 역할은 systemd가 맡고 있습니다.

target 의존성 그래프로 부팅하기

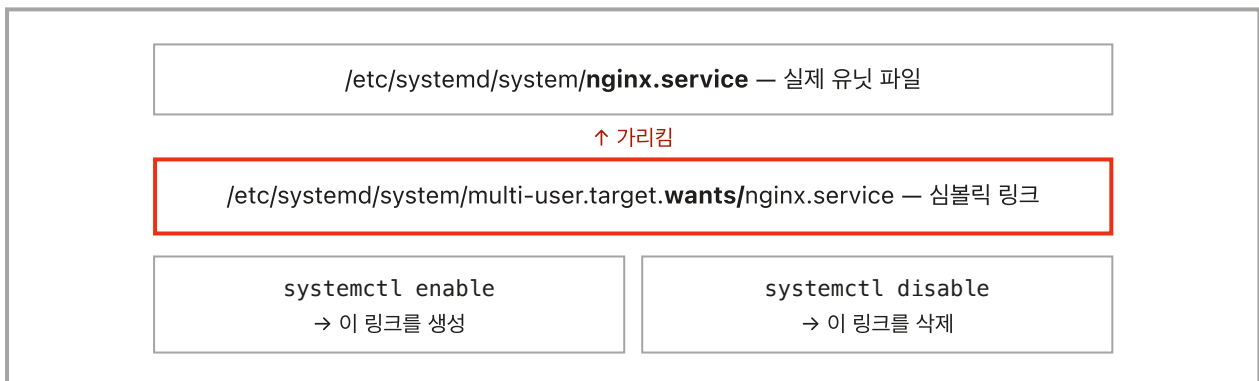
systemd는 순서를 스크립트에 하드코딩하지 않습니다. 대신 **target**이라는 동기화 지점을 여러 단계로 두고, 각 유닛이 Wants=/After=로 어느 target에 속하는지만 선언합니다. PID 1은 부팅 시점에 이 선언들을 모아 의존성 그래프를 만들고, 그래프 상 서로 막지 않는 유닛들은 **동시에** 실행합니다.



실제 그래프는 `systemctl list-dependencies myapp.service`로 트리 형태로 확인할 수 있습니다.



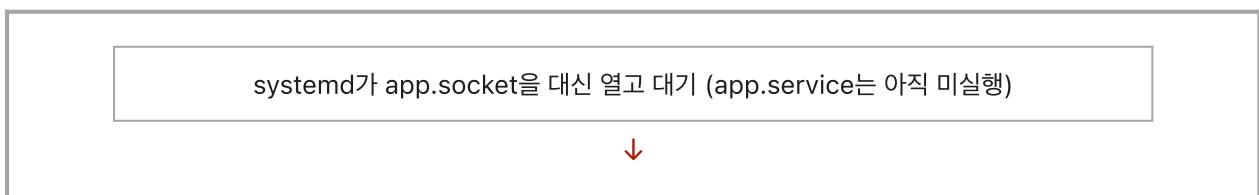
모든 target은 `.wants/` 디렉터리에 담긴 유닛들의 집합일 뿐입니다. `multi-user.target`은 텍스트 모드 부팅 완료 지점(전통 runlevel 3), `graphical.target`은 디스플레이 매니저까지 띄운 지점(runlevel 5)입니다 — 서버는 전자, 데스크톱은 후자를 기본으로 씁니다.

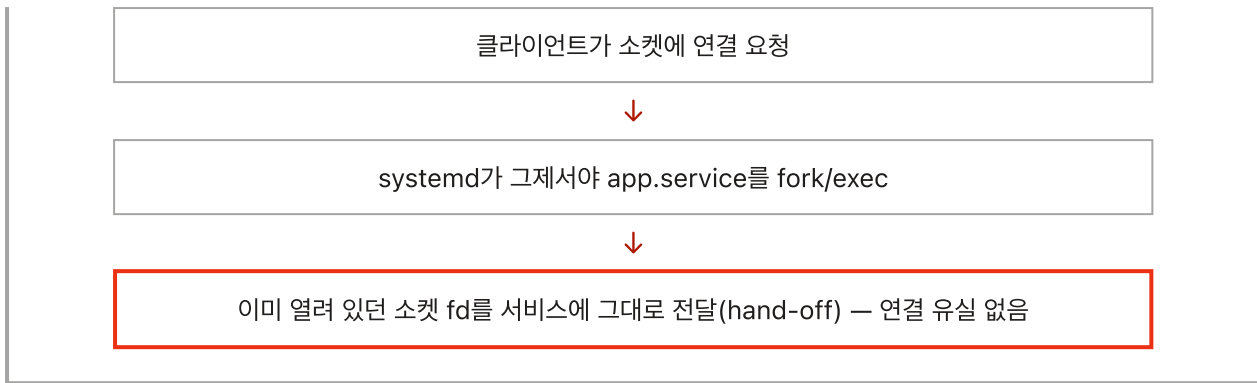


유닛 파일 자체는 지워지지 않습니다 — `enable/disable`은 오직 이 심볼릭 링크만 만들고 없앱니다. 부팅 시 `systemd`는 `multi-user.target`의 `wants/` 안에 어떤 링크가 있는지만 보고 무엇을 띄울지 결정합니다.

소켓 활성화 (Socket Activation)

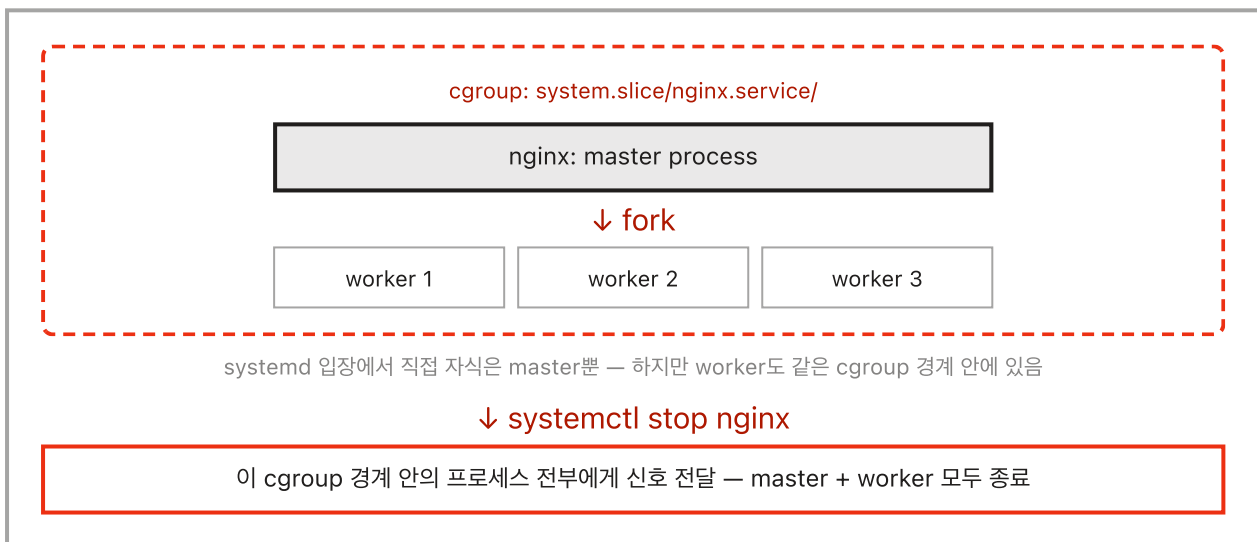
서비스가 네트워크로 요청을 받으려면 먼저 포트(또는 유닉스 소켓 파일)를 `bind`하고 연결을 받아들일 준비 (`listen`) 상태로 만들어야 합니다. 보통은 서비스 프로그램 자신이 이 작업을 합니다. `systemd`는 이 `bind+listen`을 서비스 대신 **자신이 먼저 해두고**, 실제 서비스 프로그램(`fork/exec`)은 나중에 띄울 수 있습니다. 덕분에 의존하는 서비스들이 서로의 기동 순서를 기다릴 필요가 없어집니다: 포트는 이미 열려 있으므로 연결은 즉시 받아들여지고 커널 큐에 쌓이며, 그 뒤에 실제 프로그램이 떴서 큐에 쌓인 요청을 이어받습니다.





cgroup 기반 프로세스 추적

systemd는 서비스를 시작할 때마다 전용 cgroup을 만들어 그 안에서 프로세스를 실행합니다. 서비스가 fork로 자식을 여러 개 만들거나 스스로 이중 fork(daemonize)를 해도, 커널이 관리하는 cgroup 소속 정보만 보면 그 프로세스가 "어느 서비스 소속인지" 항상 정확히 알 수 있습니다. 전통적인 init은 PID 하나만 기억했기 때문에, 서비스가 daemonize하면 추적을 놓치기 쉬웠습니다.



systemctl status nginx 출력의 CGroup: 줄이 바로 이 경로입니다.

02

systemd

systemd는 PID 1로 동작하며 서비스 관리, 로깅(journald), 장치 관리(udev 연동), 마운트, 타이머, 네트워크 이름(hostname) 등을 하나의 유닛 모델로 통합한 init 시스템입니다.

유닛(Unit)

systemd가 관리하는 대상은 모두 **유닛**이라는 텍스트 파일로 정의됩니다. 유닛 파일은 보통 `/usr/lib/systemd/system/`(배포판 제공)과 `/etc/systemd/system/`(관리자 오버라이드)에 있고, `systemctl`은 이 두 경로와 런타임 경로를 합쳐서 봅니다.

.service 데몬/프로세스 하나	.socket 소켓 활성화용 대기 지점	.target 유닛들을 묶는 동기화 지점	.timer cron 대체, 시간 트리거
.mount 마운트 지점	.device udev가 인식한 장치	.path 파일/경로 변화 감지	.slice cgroup 리소스 그룹

유닛 파일은 `[Unit]`, 타입별 섹션(`[Service]` 등), `[Install]` 세 부분으로 구성됩니다.

```
[Unit]
Description=My App
After=network.target
Requires=network.target

[Service]
ExecStart=/usr/bin/myapp
Restart=on-failure
User=myapp

[Install]
WantedBy=multi-user.target
```

[Unit] — 설명과 다른 유닛과의 의존/순서 관계

[Service] — 실행 파일, 재시작 정책, 실행 사용자 등 타입 전용 설정

[Install] — enable 시 어떤 target의 `.wants/`에 심볼릭 링크를 만들지

지시어	의미	실패 시 동작
<code>Requires=</code>	강한 의존성	대상이 실패하면 이 유닛도 중단
<code>Wants=</code>	약한 의존성	대상이 실패해도 이 유닛은 계속 진행
<code>After= / Before=</code>	시작 순서만 지정	의존성을 만들지는 않음 (병렬 시작 자체는 그대로)
<code>Conflicts=</code>	동시 실행 불가	대상이 실행되면 이 유닛은 정지됨

주의: `Requires=`는 의존성만 만들 뿐 시작 "순서"를 보장하지 않습니다. 순서까지 필요하면 `After=`를 함께 써야 합니다.

systemctl

유닛의 상태를 조회하고 제어하는 핵심 명령어입니다.

명령	설명
<code>systemctl status <unit></code>	현재 상태, 최근 로그 일부, cgroup 경로까지 요약
<code>systemctl start / stop / restart <unit></code>	지금 당장 실행/중지/재시작 (재부팅 후엔 원상복귀)
<code>systemctl enable --now / disable <unit></code>	부팅 시 자동 시작 여부와 즉시 실행을 함께 설정/해제
<code>systemctl daemon-reload</code>	유닛 파일을 수정한 뒤 systemd에게 다시 읽으라고 알림
<code>systemctl list-units --type=service</code>	현재 로드된 유닛 목록

journalctl

systemd는 각 서비스의 stdout/stderr와 커널 로그를 **journald**가 구조화된 바이너리 형식으로 `/var/log/journal/(persistent)` 또는 `/run/log/journal/(volatile)`에 기록합니다. **journalctl**은 이 로그를 조회하는 도구입니다.

명령	설명
<code>journalctl -u <unit></code>	특정 유닛의 로그만 필터링
<code>journalctl -f</code>	실시간으로 새 로그 따라가기 (tail -f와 유사)
<code>journalctl -b</code>	이번 부팅의 로그
<code>journalctl -p err</code>	지정한 심각도(err) 이상만 표시
<code>journalctl --since "1 hour ago"</code>	시간 범위로 필터링

03

컨테이너

컨테이너는 별도의 커널을 갖지 않습니다. 호스트 커널의 세 가지 기능 — **네임스페이스**(무엇을 보는가), **cgroup**(얼마나 쓸 수 있는가), **CoW 파일시스템**(어떤 파일을 보는가) — 을 조합해 "격리된 것처럼 보이는 일반 프로세스"를 만든 것입니다.

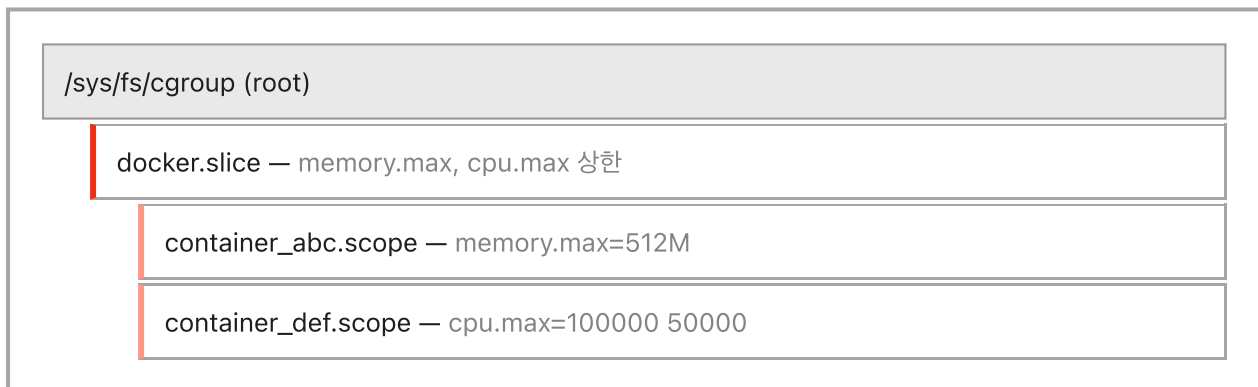
네임스페이스

네임스페이스는 프로세스가 커널의 특정 전역 리소스를 "자기만의 것"으로 보게 만드는 기능입니다. `clone()` / `unshare()` 호출에 `CLONE_NEW*` 플래그를 넘겨 생성합니다.

PID 프로세스 ID 트리 — 컨테이너 안에선 자신이 PID 1 CLONE_NEWPID	Mount 마운트 테이블 — 독립된 루트 파일시스템 뷰 CLONE_NEWNS
Network 네트워크 인터페이스, 라우팅 테이블, 포트 CLONE_NEWNET	UTS hostname, domainname CLONE_NEWUTS
IPC System V IPC, POSIX 메시지 큐 CLONE_NEWIPC	User UID/GID 매핑 — 컨테이너 root ≠ 호스트 root CLONE_NEWUSER
Cgroup 자신이 속한 cgroup 계층의 루트 위치를 감춤 CLONE_NEWCGROUP	

cgroup

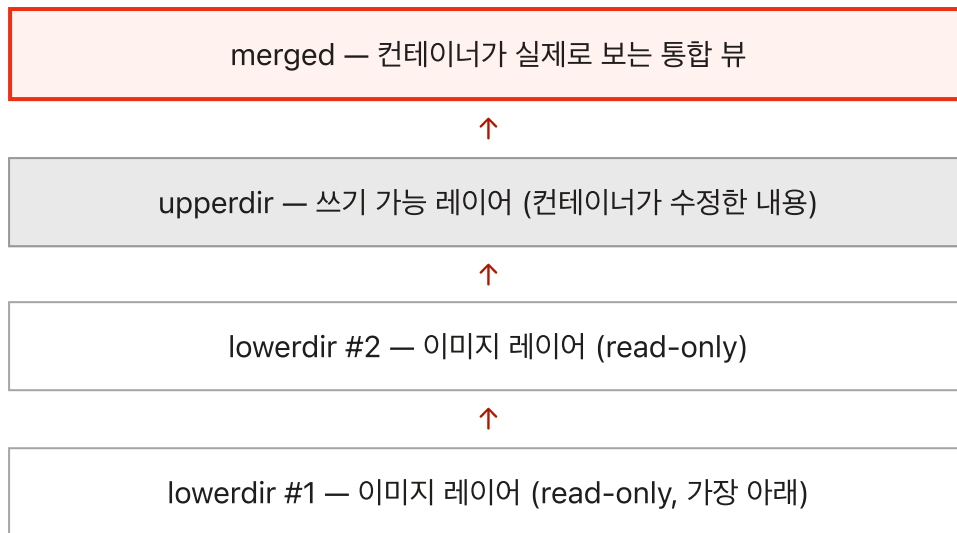
cgroup(control group)은 프로세스 그룹 단위로 CPU, 메모리, I/O 등의 자원 사용을 **제한하고 계층**합니다. 네임스페이스가 "무엇이 보이는지"를 다룬다면, cgroup은 "얼마나 쓸 수 있는지"를 다룹니다. 계층 구조는 트리로 구성되며 하위 그룹은 상위 그룹의 제한을 넘을 수 없습니다.



systemd는 자신이 관리하는 서비스/컨테이너를 slice(그룹) → scope/service(리프)로 이루어진 cgroup 트리에 자동으로 배치합니다 — 위 `systemctl status`에서 보이는 cgroup 경로가 바로 이것입니다.

CoW 파일시스템 — OverlayFS

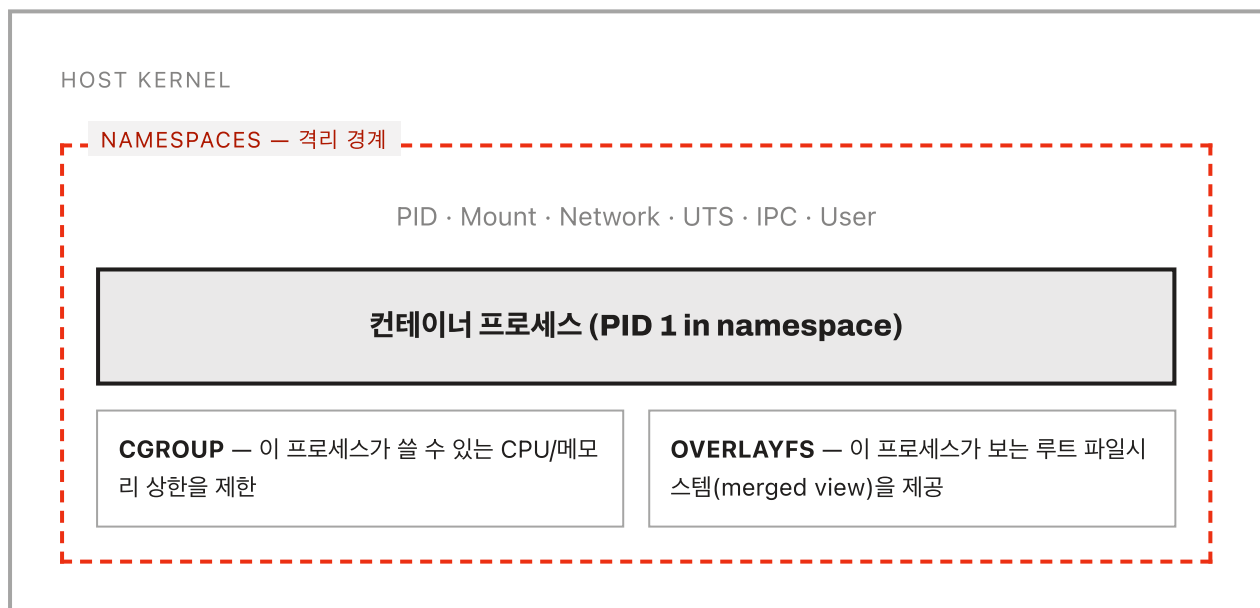
컨테이너 이미지는 여러 개의 읽기 전용 레이어로 이루어져 있고, 실행 중인 컨테이너는 그 위에 얇은 쓰기 가능 레이어 하나만 추가로 가집니다. 이것이 Copy-on-Write(CoW)입니다: 파일을 읽을 때는 위에서부터 순서대로 찾고, **수정할 때만** 하위 레이어의 파일을 쓰기 레이어로 복사한 뒤 그 복사본을 고칩니다(copy-up). 삭제는 실제로 지우지 않고 "whiteout" 표시를 남겨 하위 레이어의 파일을 가린 것처럼 보이게 합니다.



workdir는 커널이 레이어를 합치는 동안 내부적으로 쓰는 임시 작업 공간으로, upperdir와 같은 파일시스템에 있어야 합니다. 여러 컨테이너가 같은 lowerdir(이미지)를 동시에 공유하면서도 서로의 upperdir는 완전히 분리되기 때문에, 이미지 하나로 컨테이너를 여러 개 띄워도 디스크와 시작 시간을 크게 아낄 수 있습니다.

종합 아키텍처

세 기능은 서로 다른 층위에서 함께 동작해 "컨테이너"라는 하나의 개념을 만듭니다.



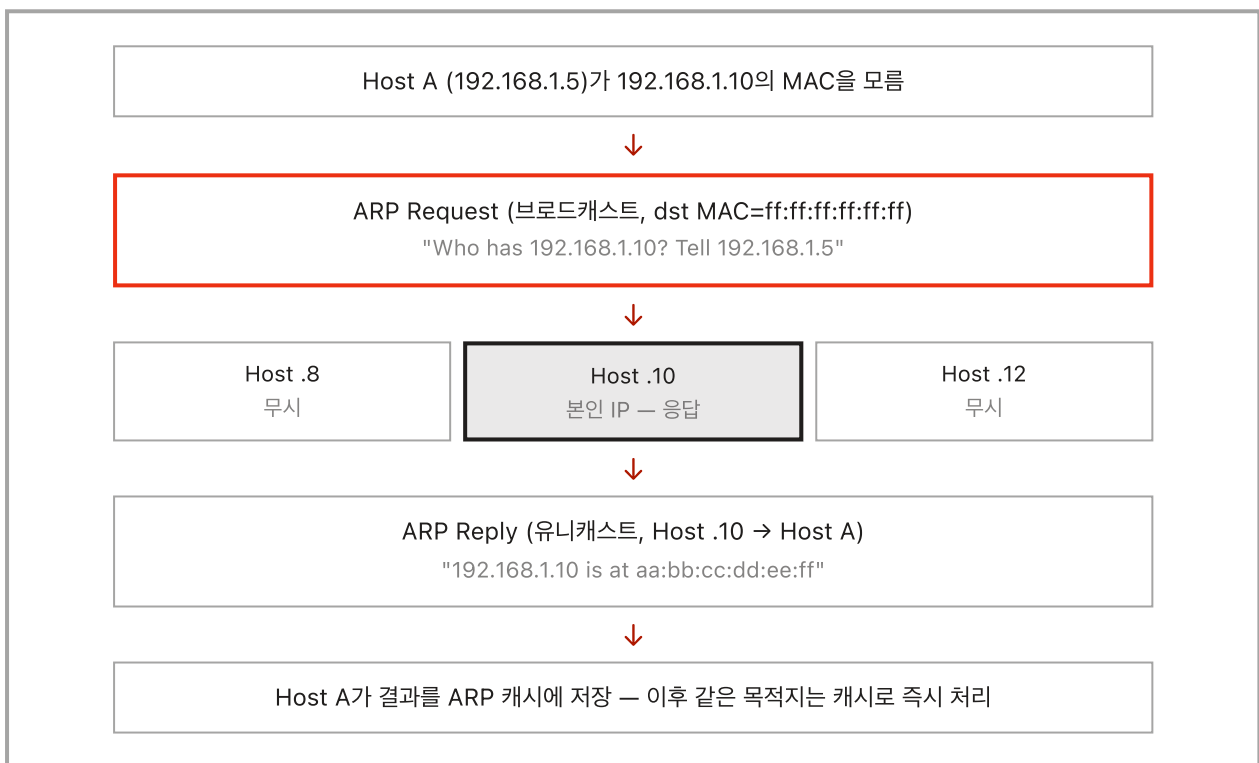
네임스페이스가 없으면 호스트의 모든 프로세스/네트워크/파일이 그대로 보이고, cgroup이 없으면 한 컨테이너가 호스트 자원을 독점할 수 있고, CoW 파일시스템이 없으면 이미지마다 매번 전체 파일시스템을 복제해야 합니다. 세 가지가 합쳐졌을 때만 "가볍고 격리된 프로세스"라는 컨테이너의 정의가 성립합니다.

네트워크 기초

컨테이너의 Network 네임스페이스가 "무엇을" 격리하는지 이해하려면, 같은 네트워크 안에서 주소가 실제로 어떻게 찾아지고(ARP) 나뉘는지(CIDR)를 알아야 합니다.

ARP — IP 주소를 MAC 주소로 바꾸기

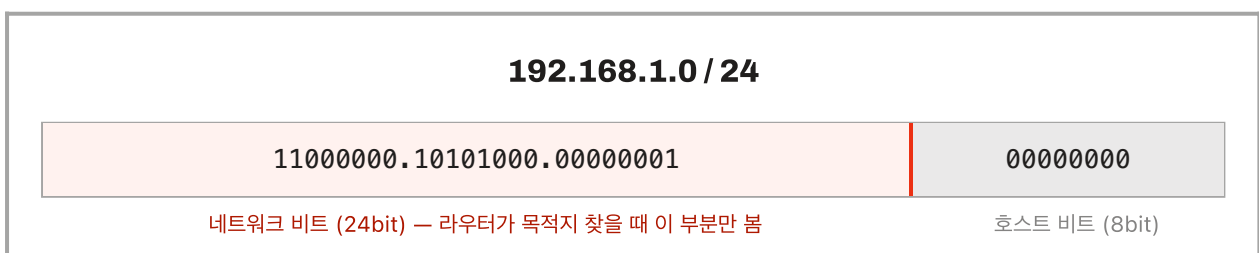
같은 이더넷 세그먼트 안에서 실제로 프레임을 전달하는 것은 IP 주소가 아니라 **MAC** 주소입니다. 호스트는 목적지 IP는 알아도 그 MAC 주소는 모를 수 있는데, **ARP(Address Resolution Protocol)**가 이 IP → MAC 매핑을 찾아줍니다.



캐시는 일정 시간(보통 수십 초~분 단위) 후 만료됩니다. 리눅스에서는 `ip neigh`(또는 구식 `arp -n`)로 현재 캐시를 확인할 수 있습니다.

CIDR — IP 주소를 네트워크/호스트로 나누기

CIDR(Classless Inter-Domain Routing) 표기법은 주소/프리픽스길이 형태로, 32비트 IPv4 주소 중 앞에서부터 몇 비트가 "네트워크"를 가리키는지를 나타냅니다. 나머지 비트는 그 네트워크 안의 개별 호스트를 구분하는 데 씁니다.



서브넷 마스크로 쓰면 255.255.255.0과 동일

프리픽스	서브넷 마스크	호스트 비트	사용 가능한 호스트 수
/16	255.255.0.0	16	65,534
/24	255.255.255.0	8	254
/26	255.255.255.192	6	62
/30	255.255.255.252	2	2

사용 가능한 호스트 수 = $2^{\text{호스트 비트}} - 2$ (네트워크 주소 1개와 브로드캐스트 주소 1개는 호스트에 할당하지 않음). 프리픽스가 클수록 (/24 → /26) 네트워크는 더 잘게 나뉘고, 네트워크당 호스트 수는 줄어듭니다.

라우팅 — 리눅스 관점

목적지가 같은 네트워크(같은 서브넷)에 있으면 ARP로 바로 MAC을 찾아 전달하면 되지만, 다른 네트워크에 있으면 **라우팅 테이블**을 보고 어느 다음 홉(next hop)으로 넘길지 결정해야 합니다. 리눅스 커널은 패킷을 보낼 때마다 이 테이블에서 목적지 IP와 가장 길게 일치하는 항목(longest prefix match)을 고릅니다.

IP ROUTE (예시)

```
192.168.1.0/24 dev eth0 scope link  
default via 192.168.1.1 dev eth0
```

패킷의 목적지 IP를 라우팅 테이블과 대조



목적지가 같은 서브넷
(192.168.1.0/24와 일치)

그 외 모든 목적지
(더 구체적으로 일치하는 규칙 없음)



ARP로 대상의 MAC을 찾아 직접 전달

default 라우트(192.168.1.1)로 전달 — 그 게이트웨이가 다시 판단

scope link는 게이트웨이 없이 직접 붙어 있다는 뜻이고, default는 더 구체적인 규칙이 없을 때 쓰는 최후의 경로입니다. IP 포워딩이 켜진(/proc/sys/net/ipv4/ip_forward=1) 리눅스 호스트는 이 과정을 반복해 라우터 역할도 할 수 있습니다.

포트

같은 IP를 가진 호스트 하나가 동시에 여러 서비스를 제공하거나 여러 연결을 유지할 수 있는 것은 **포트 (port)** 번호 덕분입니다. 포트는 0~65535 범위의 16비트 숫자로, 같은 IP 안에서 어떤 프로세스에게 데이터를 전달할지 구분합니다.

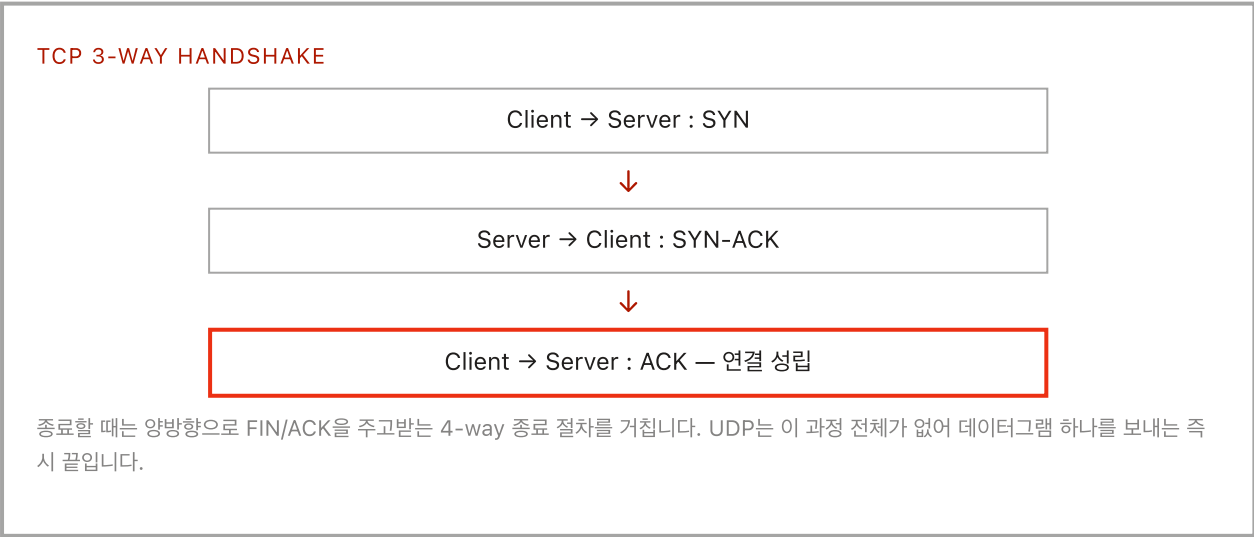
포트	서비스
22	SSH
53	DNS
67 / 68	DHCP (서버 / 클라이언트)
80 / 443	HTTP / HTTPS
2049	NFS

1~1023번은 "well-known 포트"로 관례상 root 권한이 있어야 바인드할 수 있습니다. 실제로 하나의 연결을 구분하는 단위는 포트 하나가 아니라 (출발지 IP, 출발지 포트, 목적지 IP, 목적지 포트)로 이루어진 4-tuple입니다 — 같은 서버의 80번 포트에 여러 클라이언트가 동시에 접속해도 이 4-tuple이 달라 서로 구분됩니다.

TCP/UDP

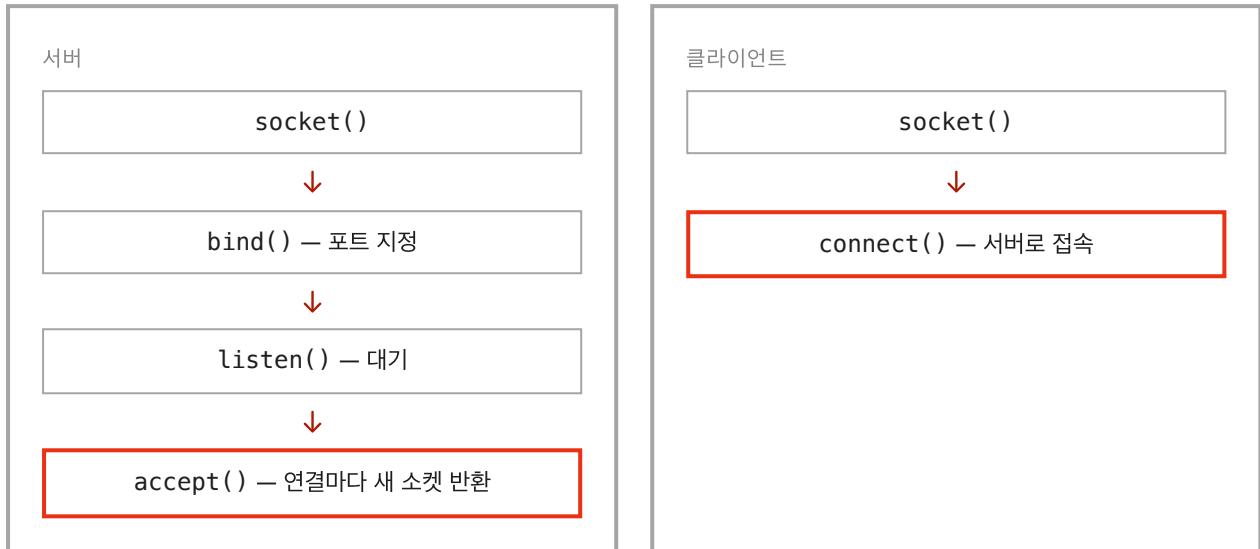
포트 위에서 실제로 데이터를 주고받는 두 가지 대표적인 전송 계층 프로토콜입니다.

	TCP	UDP
연결	연결 지향 (핸드셰이크 필요)	비연결형 (그냥 보냄)
순서/신뢰성	순서 보장, 재전송으로 신뢰성 보장	보장 없음
오버헤드	상대적으로 큼	작음, 지연에 민감한 용도에 적합
대표 사용처	HTTP, SSH, NFS(v4)	DNS 조회, DHCP, 스트리밍



소켓

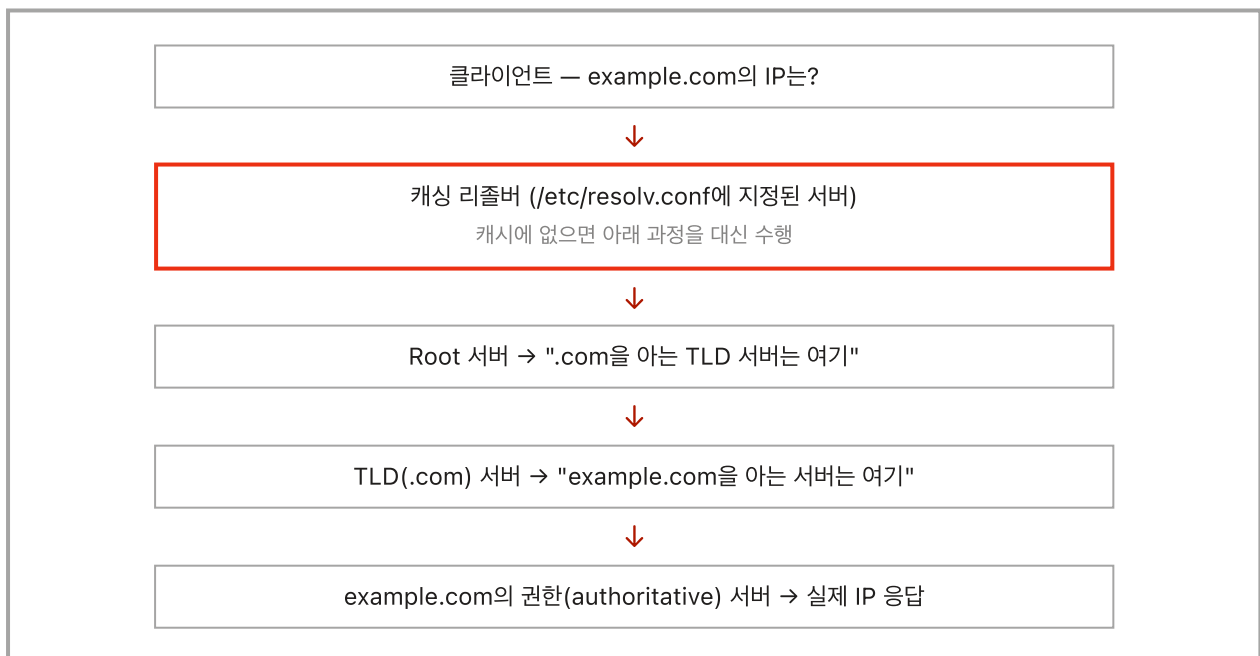
소켓(socket)은 네트워크 통신의 끝점을 가리키는 커널 객체이자, 프로그램이 그것을 다루는 파일 디스크립터입니다. 서버와 클라이언트는 각각 다른 시스템 호출 순서를 밟습니다.



이후 양쪽 모두 같은 파일 디스크립터에 `read()/write()`를 쓰듯 데이터를 주고받습니다. `AF_INET` 소켓은 네트워크(IP+포트)를 쓰고, `AF_UNIX` 소켓은 같은 호스트 안에서 파일시스템 경로를 주소로 쓰는 로컬 전용 소켓입니다.

DNS

사람이 읽는 도메인 이름을 IP 주소로 바꾸는 과정입니다. 클라이언트는 보통 직접 루트 서버까지 묻지 않고, 리졸버가 대신 반복 질의(iterative query)를 수행합니다.



리눅스에서는 `/etc/hosts`를 DNS보다 먼저 확인하며, 어떤 순서로 무엇을 조회할지는 `/etc/nsswitch.conf`의 `hosts:` 항목이 정합니다. 주로 UDP 53번을 쓰지만 응답이 512바이트를 넘으면 TCP 53번으로 전환됩니다.

SSH

암호화된 채널로 원격 로그인/명령 실행을 제공하는 프로토콜입니다(기본 포트 22). 접속 시 두 단계를 거칩니다.

1. 서버 신원 확인

서버가 자신의 호스트 키를 제시 → 클라이언트가
~/.ssh/known_hosts에 기록된 것과 비교. 처음 접속이면 "fingerprint를 신뢰하겠느냐"고 묻습니다.

2. 사용자 인증 (공개키 방식)

클라이언트가 개인키로 서명 → 서버는
~/.ssh/authorized_keys에 있는 공개키로 그 서명을 검증. 통과하면 셸이 열립니다.

비밀번호 인증도 가능하지만, 공개키 방식은 개인키가 네트워크로 전송되지 않고 서명만 오가기 때문에 더 안전합니다.

NFS

NFS(Network File System)는 원격 서버의 디렉터리를 로컬 파일시스템처럼 마운트해 쓰는 프로토콜입니다. 애플리케이션은 로컬 파일과 똑같이 `read()`/`write()`를 호출하지만, 커널의 VFS 계층이 그 요청을 네트워크 너머 서버로 전달합니다.

서버: /etc/exports에 /data를 클라이언트 대역에 공개(export)



클라이언트: `mount server:/data /mnt/data`



이후 /mnt/data에 대한 모든 파일 연산이 RPC로 서버에 전달됨

NFSv3는 서버가 클라이언트 상태를 기억하지 않는 stateless 설계였고, NFSv4는 파일 잠금(lock)과 세션 상태를 서버가 직접 관리하는 stateful 설계로 바뀌었습니다.

DHCP

호스트가 부팅 시 자신의 IP 주소조차 모를 때, 네트워크 설정(IP, 서브넷 마스크, 게이트웨이, DNS 서버)을 자동으로 받아오는 프로토콜입니다. 4단계 교환을 **DORA**라고 부릅니다.

Discover — 클라이언트가 브로드캐스트로 "DHCP 서버 어디 있나요?"



Offer — 서버가 "이 IP를 줄 수 있어요"라고 제안



Request — 클라이언트가 그 IP를 쓰겠다고 요청(브로드캐스트로, 다른 서버들도 보게)



Ack — 서버가 확정 응답, 임대(lease) 시작

임대는 영구적이지 않고 시간 제한(lease time)이 있어, 절반쯤 지나면 클라이언트가 같은 서버에 조용히 갱신(renew)을 요청합니다.

Modernist design system 기준으로 작성됨.